

DBMS • FINAL IMPLEMENTATION

ENTERPRISE PARCEL MANAGEMENT SYSTEM

Database Management System — Final Implementation

PROJECT TEAM

Shivam Tiwari • V. Akhil • Balaji Patil



One platform. Every parcel event.

A centralized, database-driven platform manages the complete parcel lifecycle—from booking through delivery and historical records.



Customer & Parcel

Profiles, bookings and searchable parcel records.

Shipment & Warehouse

Movement planning, warehouse handoffs and status.

Delivery & Payment

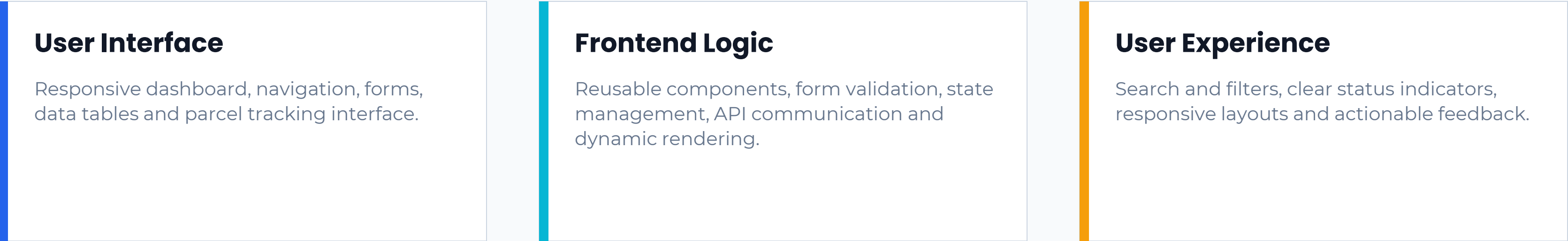
Assignments, confirmation and transaction records.

Tracking & Reporting

Live history, audit trail and operational views.

A responsive operating surface

React + HTML + CSS + JavaScript • component-based, validated and API-connected.



Six focused workspaces

Role-aware modules keep common tasks fast while preserving a consistent interaction model.

Dashboard

Parcel stats • shipment overview • quick actions

Customer

Registration • profile • booking • tracking

Parcel

Create • view • update • search and filter

Shipment

Details • status • warehouse movement

Delivery

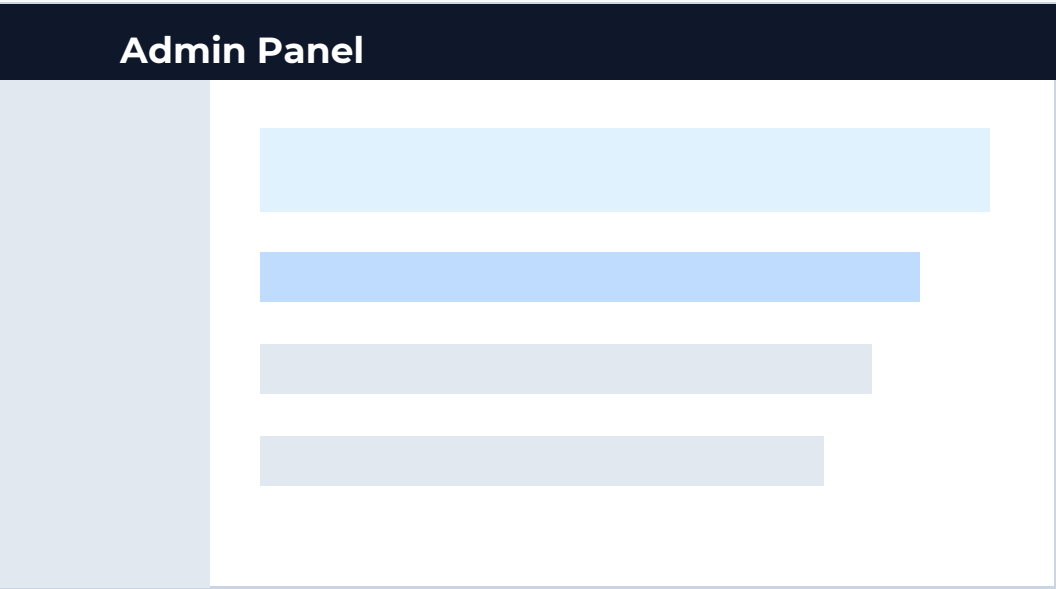
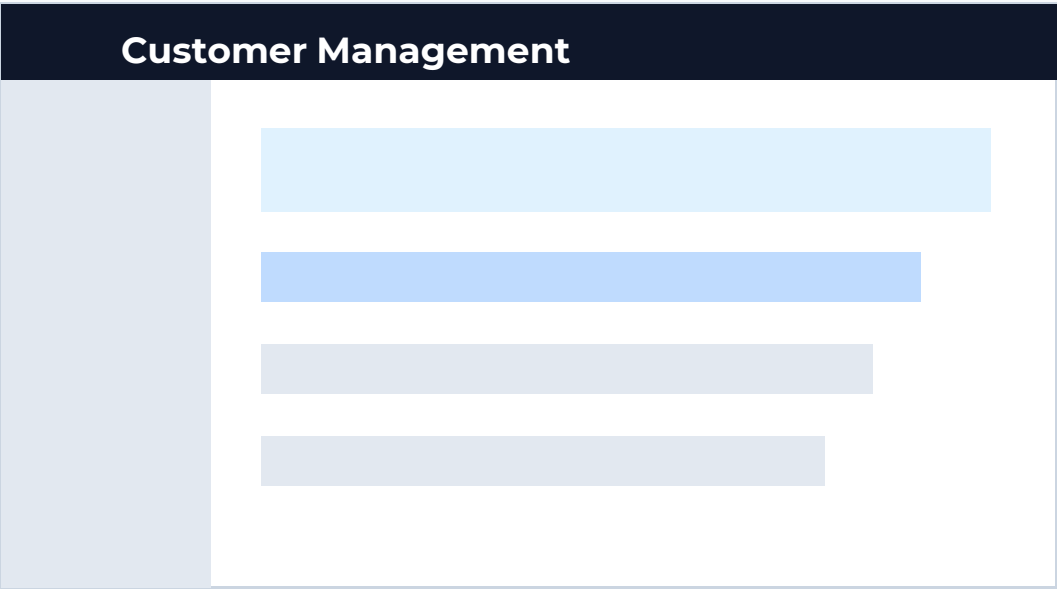
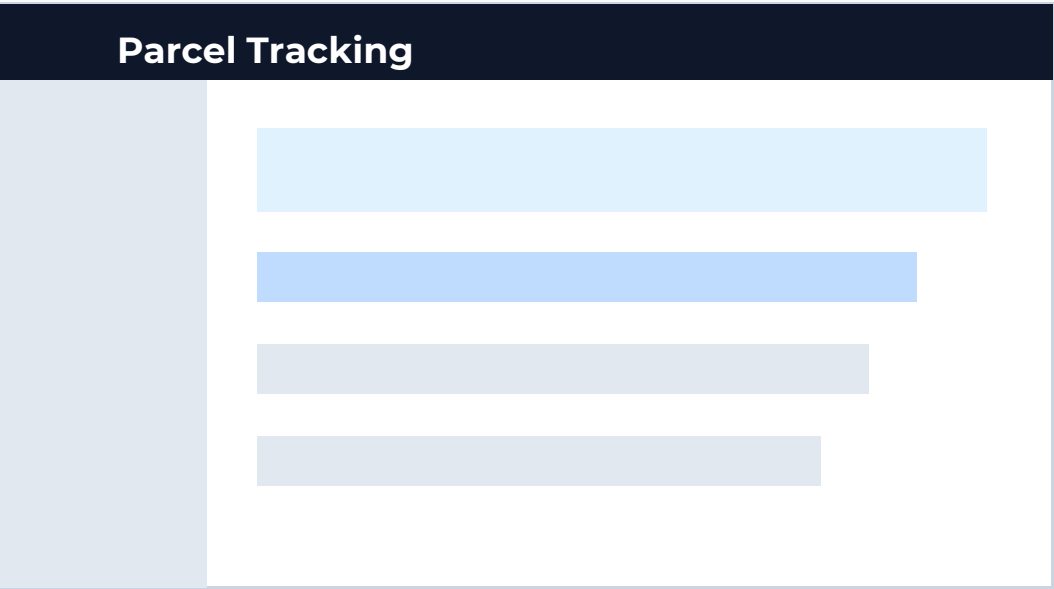
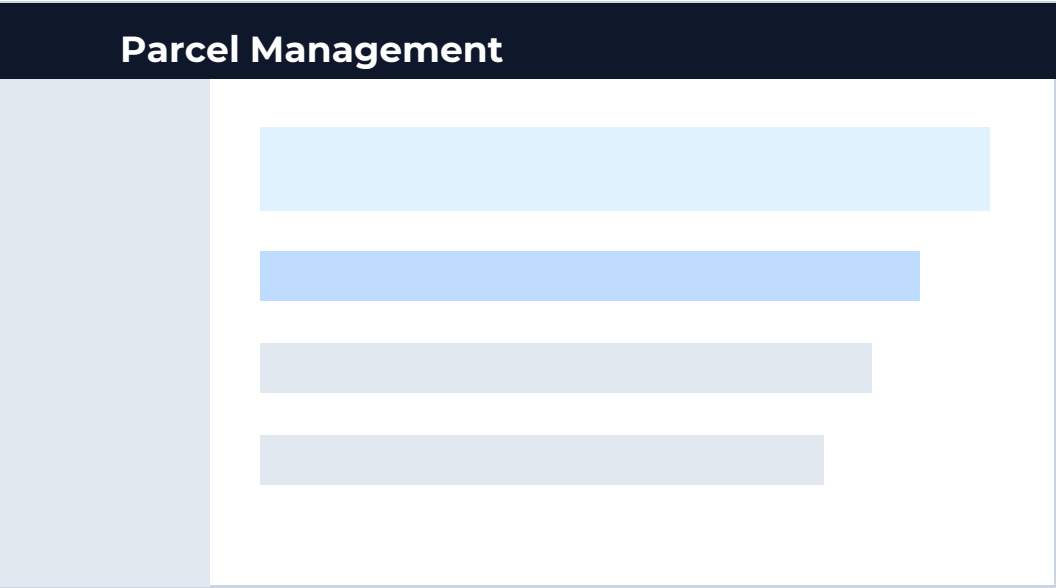
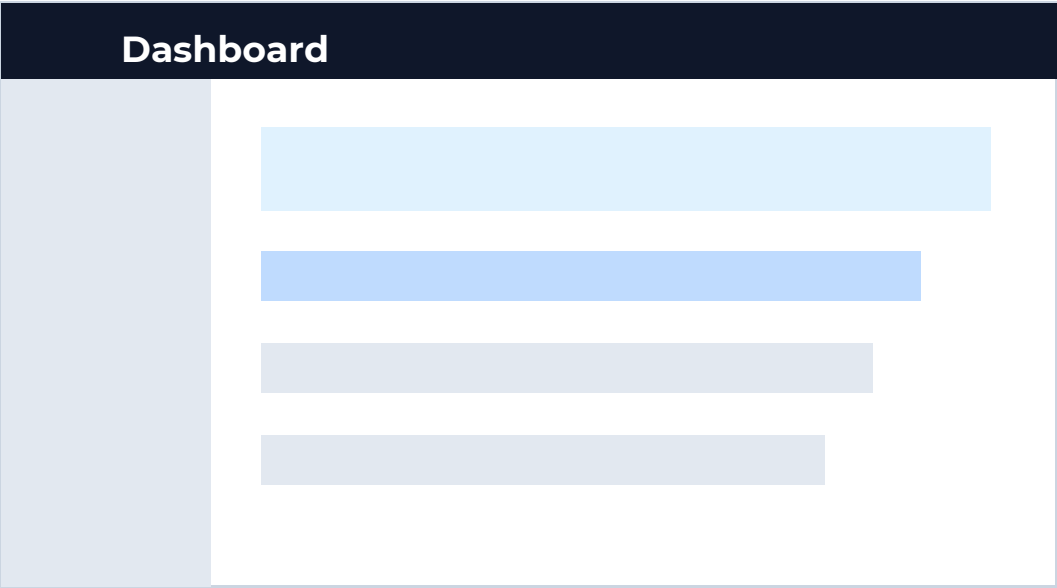
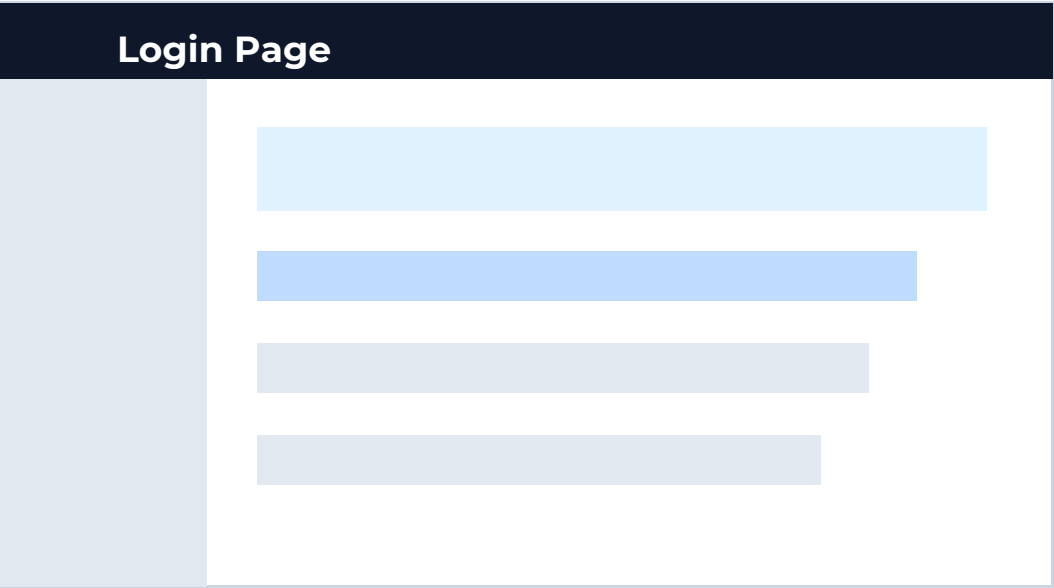
Assignment • confirmation • status

Admin

Users • warehouses • parcels • reports

A consistent enterprise UI system

Representative implementation views shown as editable interface mockups—not fabricated test results.



Frontend → Backend → Database

A traceable request-response path with validation and persistence at every boundary.

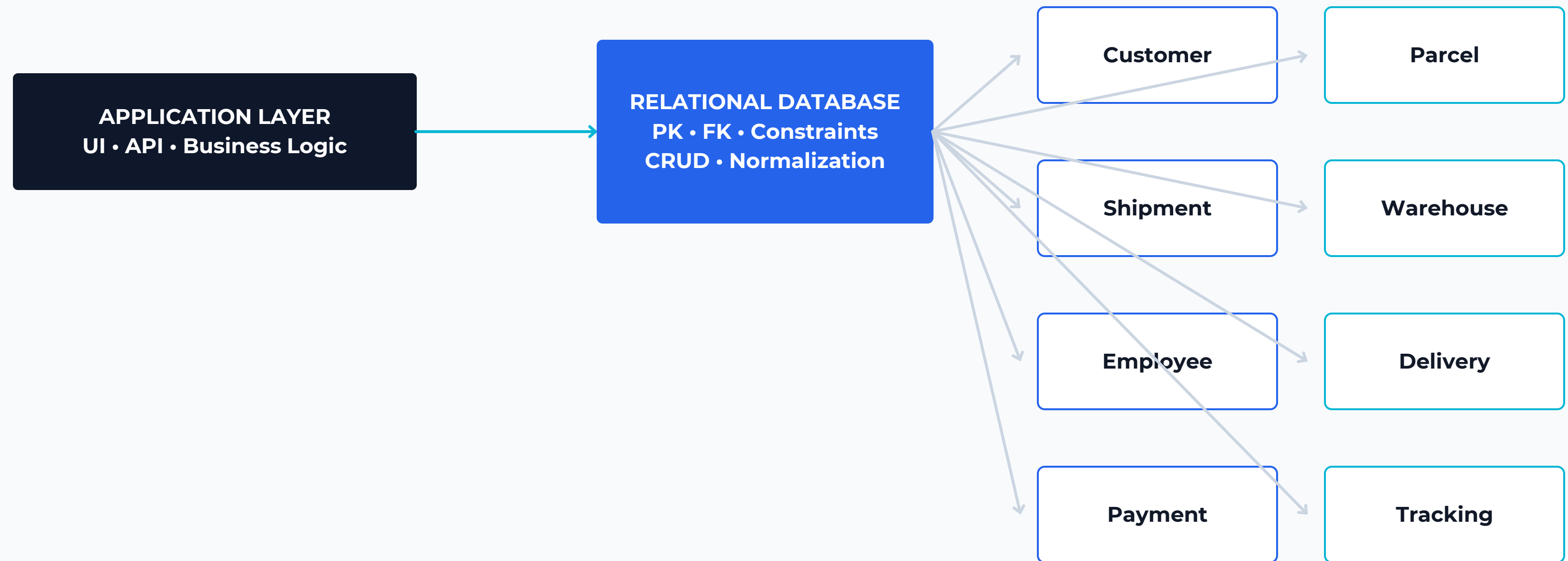


Example: “Track Parcel”

GET /parcels/{trackingId} → validate request → SELECT Parcel + latest Tracking event → serialize response → update tracking timeline

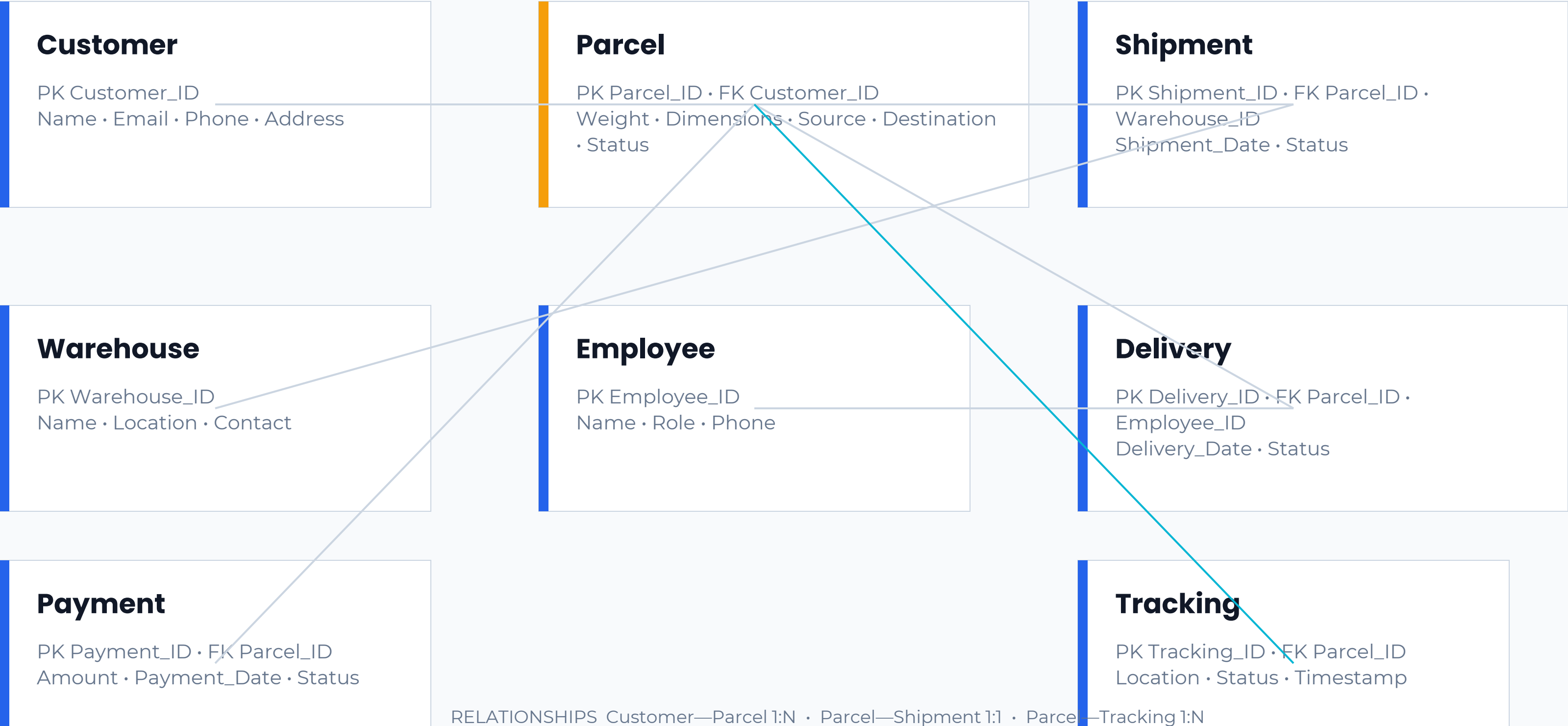
Relational by design

Normalized entities, explicit keys and enforceable integrity create a dependable operational record.



Entities, keys and cardinality

Core schema view; identifiers are highlighted first, followed by operational attributes.



Each table has one clear responsibility

A compact catalog supports implementation review and schema ownership.

Customer	Stores customer information
Parcel	Stores parcel details
Shipment	Manages shipment information
Warehouse	Stores warehouse information
Employee	Stores staff and delivery-agent information
Delivery	Tracks delivery assignments and status
Payment	Stores payment and transaction information
Tracking	Stores movement and status history

Identifiers create the data spine

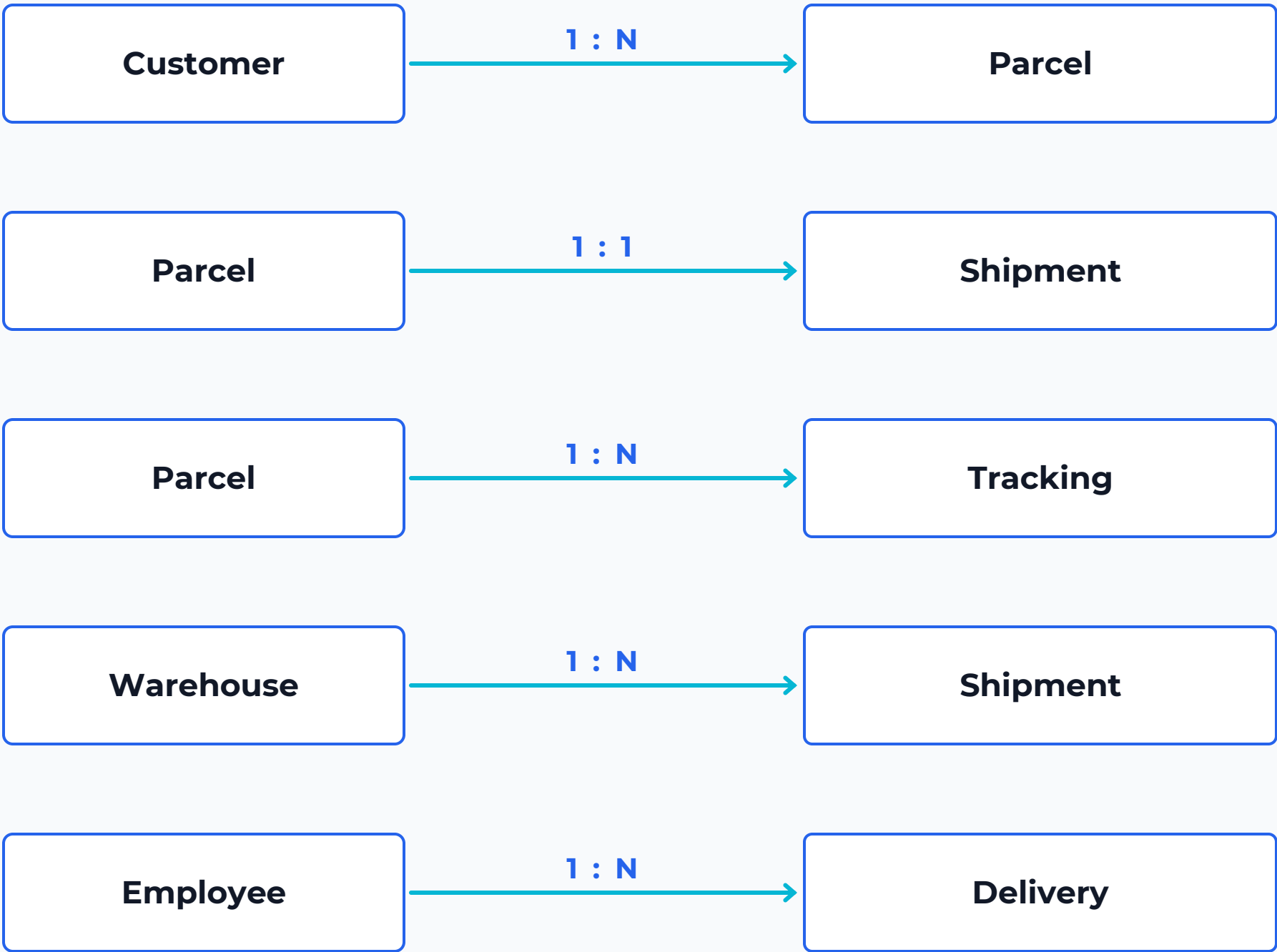
Primary keys establish identity. Foreign keys preserve relationships and referential integrity.

PRIMARY KEY

Unique row identity
Examples: Customer_ID, Parcel_ID

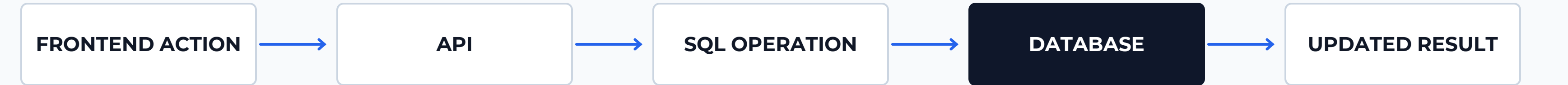
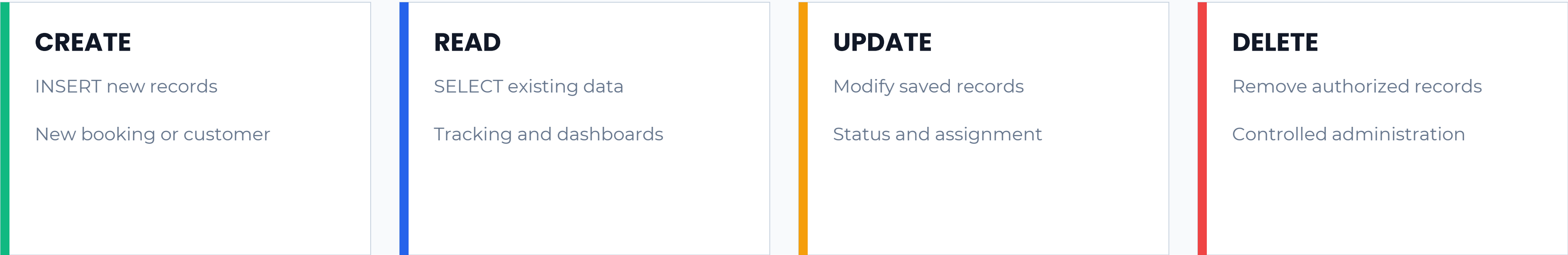
FOREIGN KEY

Cross-table reference
Parcel.Customer_ID → Customer.Customer_ID



Four operations. One consistent path.

Every UI action is validated by the API, translated to SQL and returned as an updated application state.



Queries that power the workflow

01 • Parcel by ID

```
SELECT * FROM Parcel  
WHERE Parcel_ID = ?;
```

02 • Delivered parcels

```
SELECT * FROM Parcel  
WHERE Status = 'Delivered';
```

03 • Customer + parcel

```
SELECT c.Name, p.Parcel_ID, p.Status  
FROM Customer c JOIN Parcel p  
ON p.Customer_ID = c.Customer_ID;
```

04 • Update status

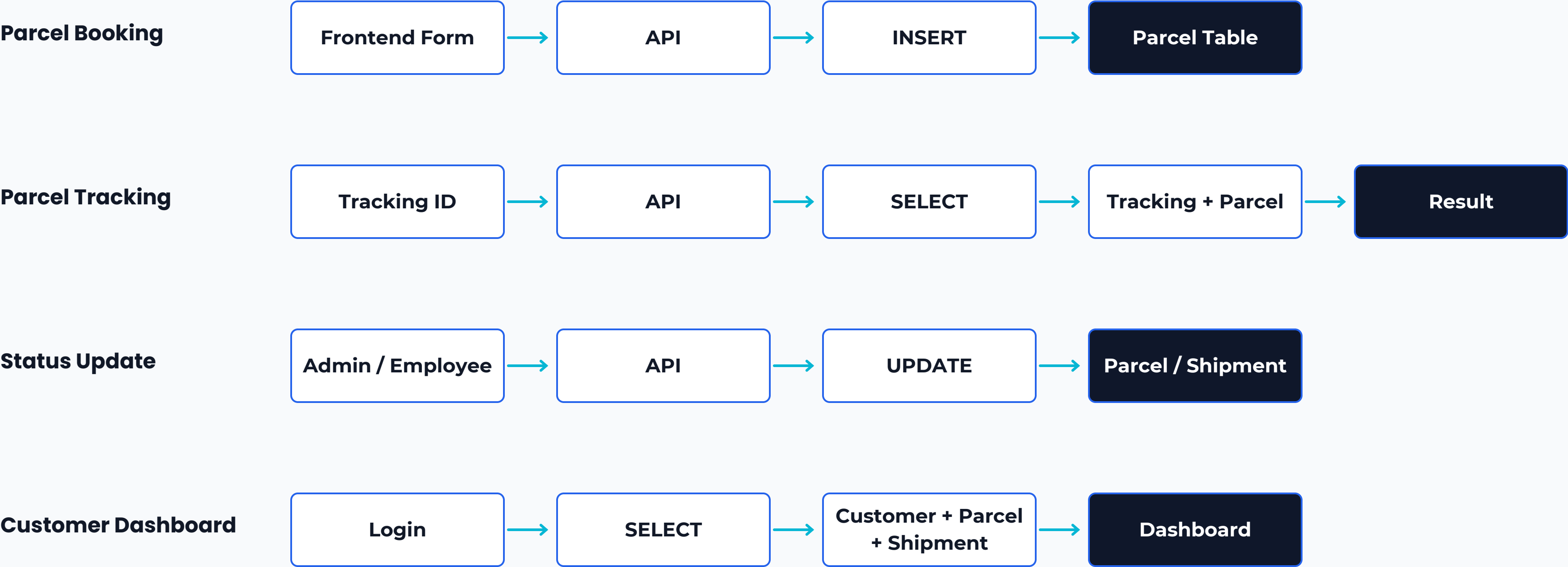
```
UPDATE Parcel SET Status = ?  
WHERE Parcel_ID = ?;
```

05 • Count by status

```
SELECT Status, COUNT(*) AS Total  
FROM Parcel GROUP BY Status;
```

UI intent becomes a database operation

Four core journeys show how screens, APIs, SQL and tables remain aligned.



Core system implemented; refinement continues

Status reflects scope—not fabricated completion percentages or test claims.

COMPLETED

Database design • ER diagram • relational schema • tables and keys • CRUD • frontend structure • backend integration • core modules

CURRENTLY WORKING ON

UI refinement • validation • testing • reports • performance optimization

NEXT

Deployment • real-time tracking • notifications • analytics • route optimization



BUILT



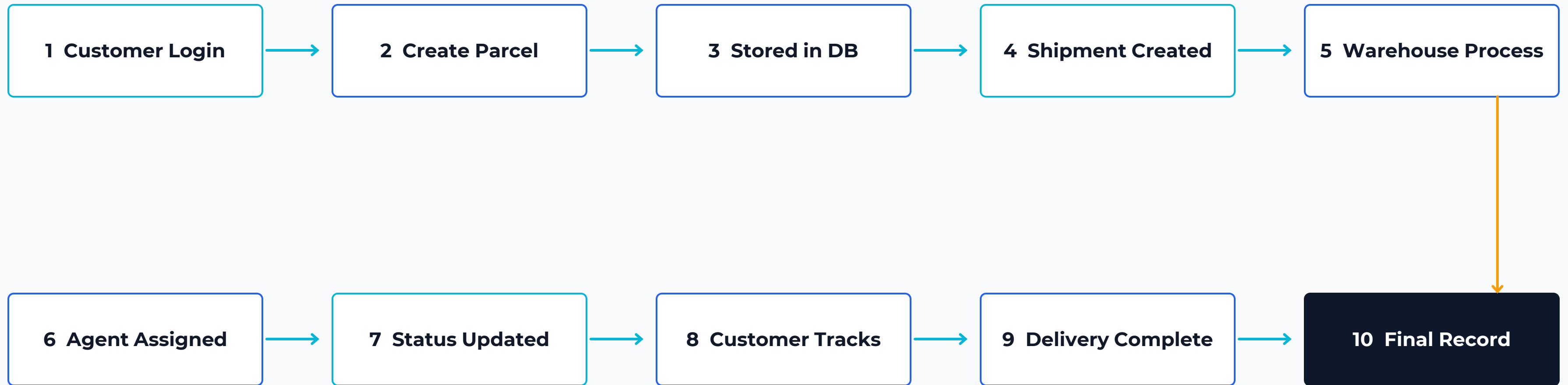
REFINE



EXTEND

From intent to immutable record

The system maintains continuity across customer action, logistics execution and final delivery history.



A complete relational workflow foundation

The system demonstrates how frontend, backend, SQL, ER design and CRUD combine into a centralized platform supporting the full parcel lifecycle.

01 Testing

02 Deployment

03 Real-Time Tracking

04 Notifications

05 Analytics

06 AI-based Route Optimization

QUESTIONS & DISCUSSION

THANK YOU

Enterprise Parcel Management System

Shivam Tiwari · V. Akhil · Balaji Patil

